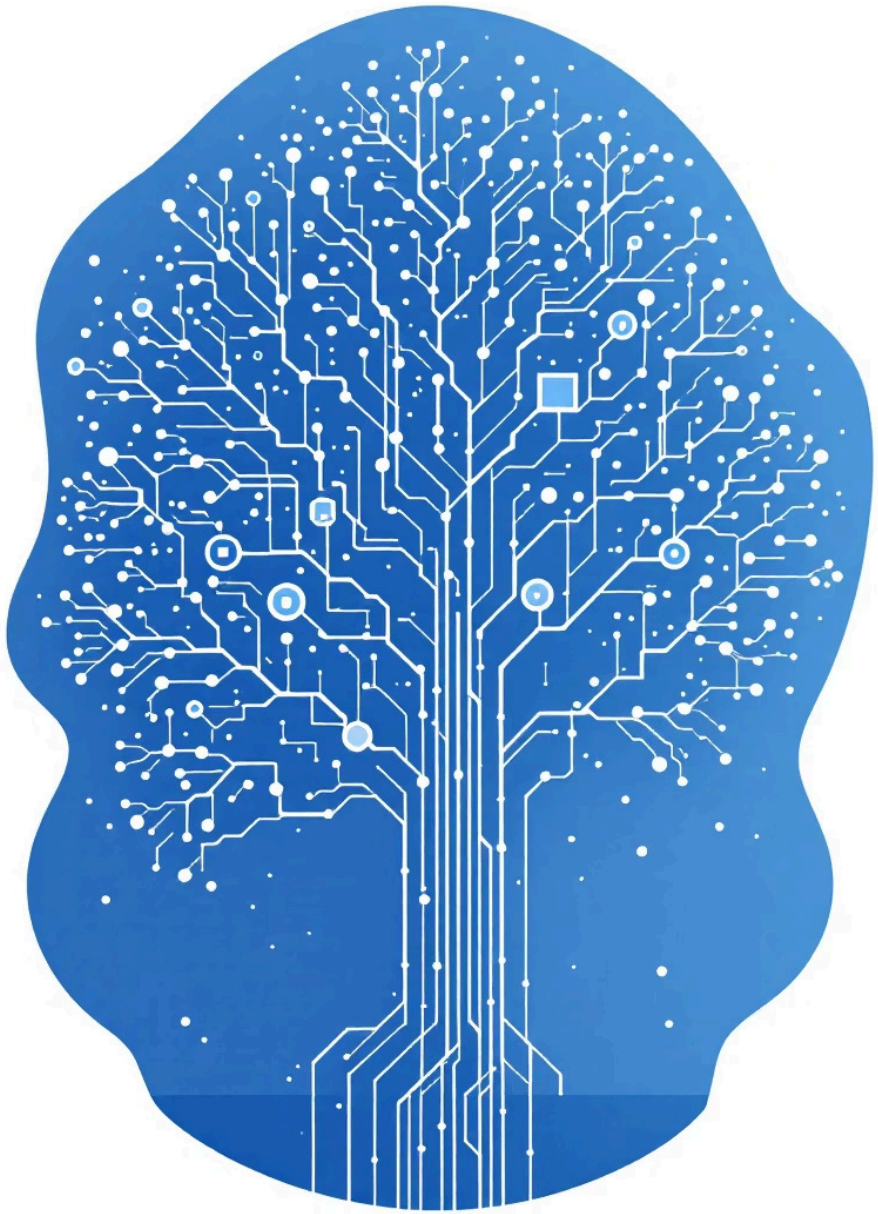


Veri Yapıları ve Algoritmalar

Ağaçlar (Trees)

Doç. Dr. Ömer GÜLEÇ

www.pau.edu.tr/omer

Pamukkale Üniversitesi
Yönetim Bilişim Sistemleri

Ağaçlar (Trees)

Hiyerarşik Yapı

Verilerin bellekte ağaç görünümlü şekilde sanal olarak bağlanması

Kullanım Alanları

Veritabanı dizinleme, dosya sistemleri, oyun hamleleri, kodlama ve veri arama

Performans Avantajı

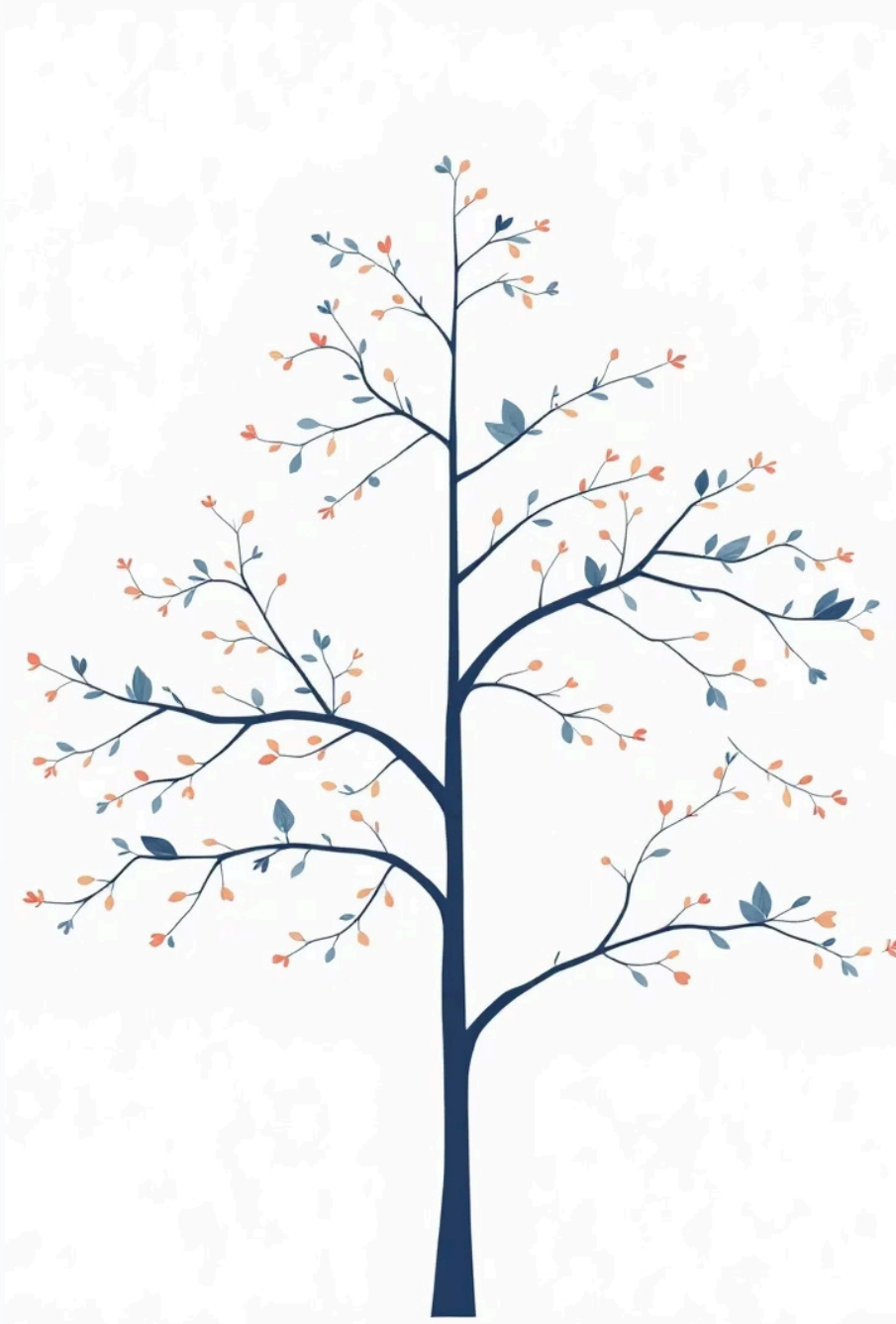
n=1000 için bağlı listede 1000 birim, ağaçta <10 birim işlem

Avantajlar

- O(log2N) karmaşıklıkla hızlı arama
- Recursive fonksiyonlarda kolay tasarım
- Runtime'da sağladığı kolaylık

Dezavantajlar

- Daha fazla bellek ihtiyacı
- Birçok işaretçi ve değişken



Nerelerde Kullanılır?

1. Dosya sistemleri (file systems)

- Bilgisayardaki klasör ve dosya yapısı ağaç şeklindedir.
  - Örneğin:

```
C:/
 ├─── Users/
 │    ├─── Omer/
 │    │    ├─── Documents/
 │    │    └─── Pictures/
 └─── Windows/
```

- Klasörler “düğümler” (nodes), alt klasörler “alt ağaçlar” (subtrees) gibidir.

2. Veritabanları ve arama işlemleri

- **İkili arama ağacı (Binary Search Tree)** ve türevleri (**AVL**, **Red-Black Tree**) verileri hızlı aramak, eklemek, silmek için kullanılır.
- **B-Tree** ve **B+Tree** yapıları, veritabanlarında (ör. MySQL, PostgreSQL) indeksleme için kullanılır.
  → Böylece milyonlarca kayıtta hızlı sorgu yapılabilir.

3. Yol bulma ve grafik algoritmaları

- Bazı grafik algoritmaları (örneğin Dijkstra, Prim, Kruskal) **ağaç yapıları** veya **ağaç benzeri alt yapılar**(spanning tree) oluşturur.
- Bu yapılar, **en kısa yol**, **ağ bağlantısı**, **minimum maliyetli yapı** gibi problemlerde kullanılır.

4. Programlama dilleri ve derleyiciler

- Bir program çalıştırılmadan önce, derleyici kodu **soyut sözdizim ağacına (Abstract Syntax Tree – AST)** dönüştürür.
  - Örneğin:

```
x = 5 + 3
```

Bu ifade bir ağaç şeklinde temsil edilir:

```
    (=)
   /  \
  x   (+)
      /  \
     5    3
```

5. Veri sıkıştırma ve kodlama

- **Huffman ağacı** gibi yapılar, veri sıkıştırma algoritmalarında (ZIP, JPEG, MP3 vb.) kullanılır.
  - Daha sık geçen karakterlere kısa kodlar, nadir olanlara uzun kodlar atanır.

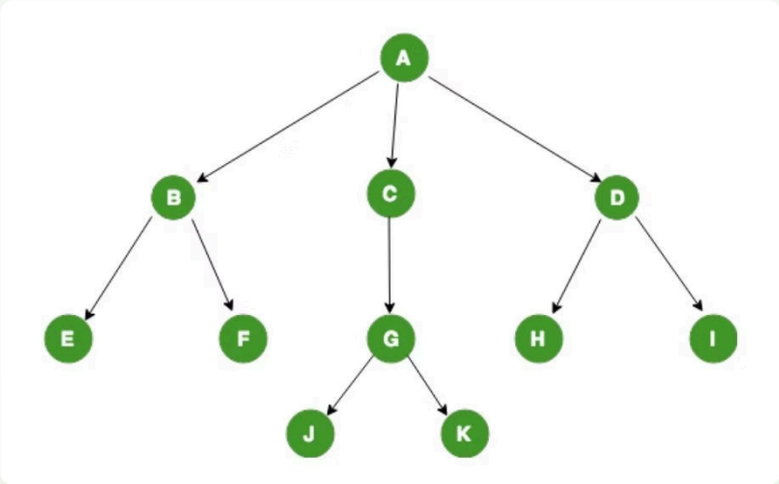
6. Yapay zeka ve karar sistemleri

- **Karar ağaçları (Decision Trees)** makine öğrenmesinde sınıflandırma veya tahmin yapmak için kullanılır.
  - Örnek: “Hava güneşliyse dışarı çık, değilse evde kal.”

7. Oyunlar ve yapay zekâ stratejileri

- Satranç veya Go gibi oyunlarda, **oyun ağaçları** olası hamlelerin dallanmış yapısını gösterir.
  - Bu ağaçlar sayesinde AI hangi hamlenin en iyi olduğunu hesaplar.

Ağaç Yapısının Temel Kavramları



Temel Bileşenler

Kök (root), kök işaretçisi, sonlu sayıda düğüm ve düğümleri birbirine bağlayan dallar

Düğüm Tipleri

Kök düğüm (root) ve yaprak düğüm (leaf) olmak üzere iki temel tip

Hiyerarşik İlişkiler

Çocuk (child), ebeveyn (parent), kardeş (sibling), ata ve torun (ancestor-descendant)

Yapısal Özellikler

Orman (forest), yol (path), derece (degree), düzey (level), derinlik (depth), yükseklik (height)

Temel Ağaç Kavramları

Kök (Root)

Ağacın en tepedeki ilk düğümü. Bir ağacın sadece **1 kök** düğümü vardır.

Öncelikle kök oluşturulur, sonra diğer düğümler eklenir.

Yaprak (Leaf)

Harici veya terminal düğüm olarak da bilinir. Yaprak düğümden daha aşağıya gidilemez.

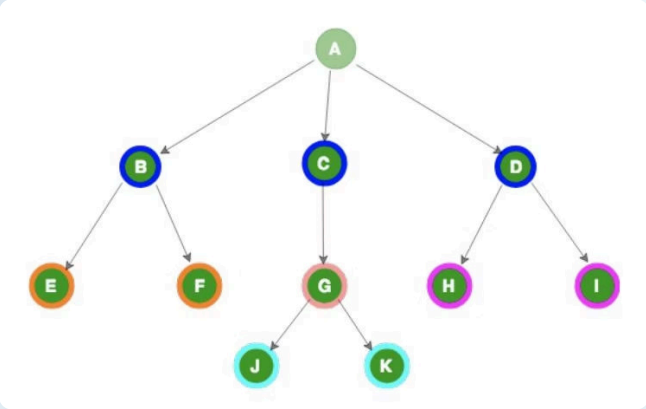
Edge (Kenar)

Düğümler arası bağlantı.

Path (Yol)

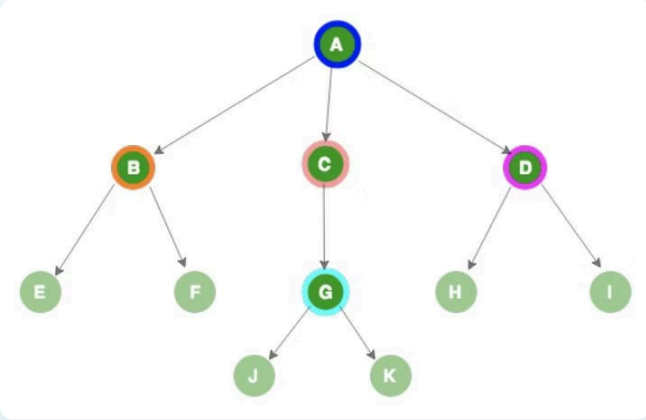
Düğümler arasındaki bağlantı dizisi.

Düğüm İlişkileri



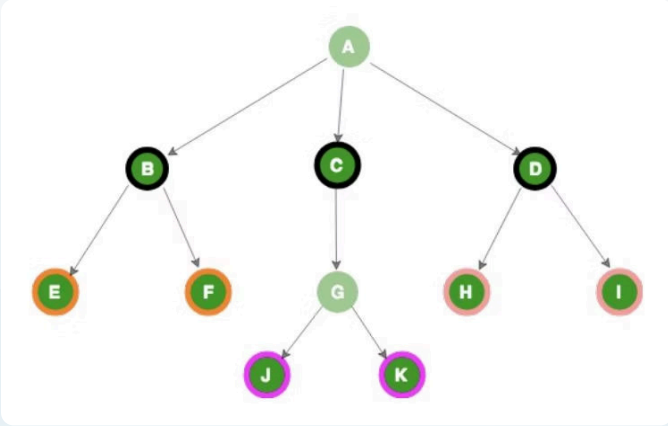
Child (Çocuk)

Bir düğüme doğrudan bağlı olan düğümler. Bir düğüme bağlı alt ağaçların kökleri.



Parent (Ebeveyn)

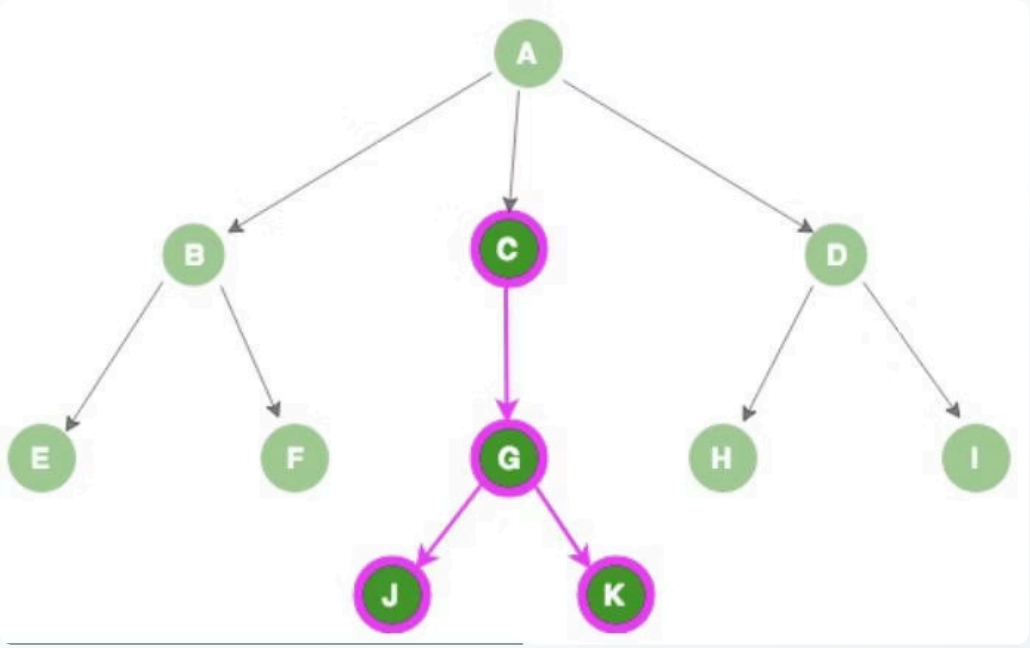
Bir düğümün üst düğümü. Her düğümün bir ebeveyni vardır (kök hariç).



Siblings (Kardeş)

Aynı parent düğüme bağlı düğüm grubu. Aynı seviyede olan düğümler.

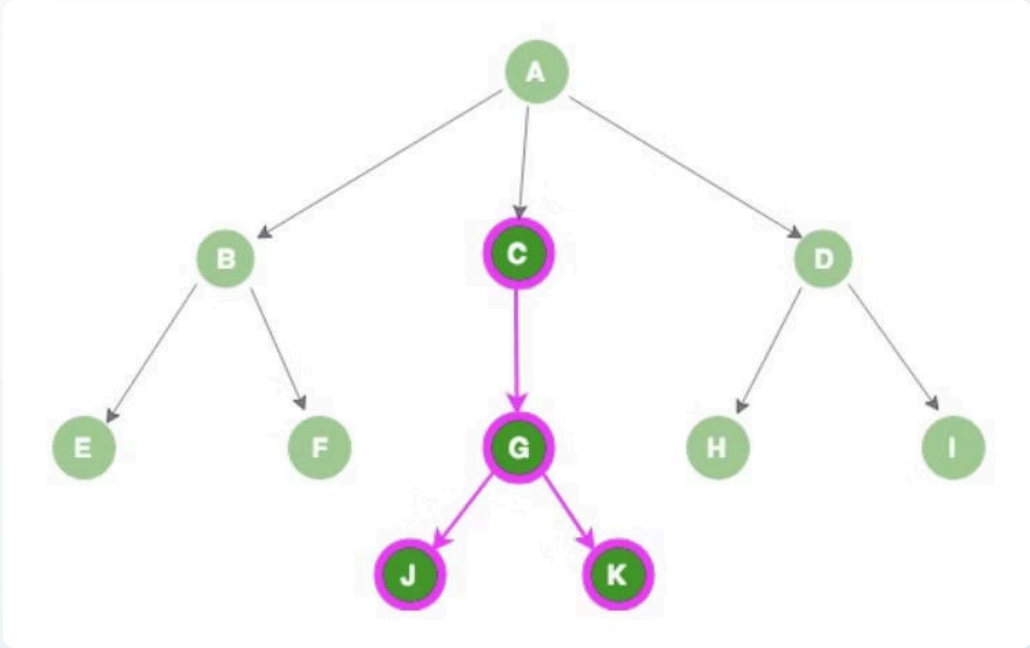
Ata ve Torun İlişkileri



Descendant (Torun)

Bir düğümden yaprak düğüme kadar olan yoldaki herhangi bir düğüm (yaprak dahil).

Bir düğüme en yakın descendant: child



Ancestor (Ata)

Bir düğümden kök düğüme kadar olan yoldaki herhangi bir düğüm (kök dahil).

Bir düğüme en yakın ancestor: parent

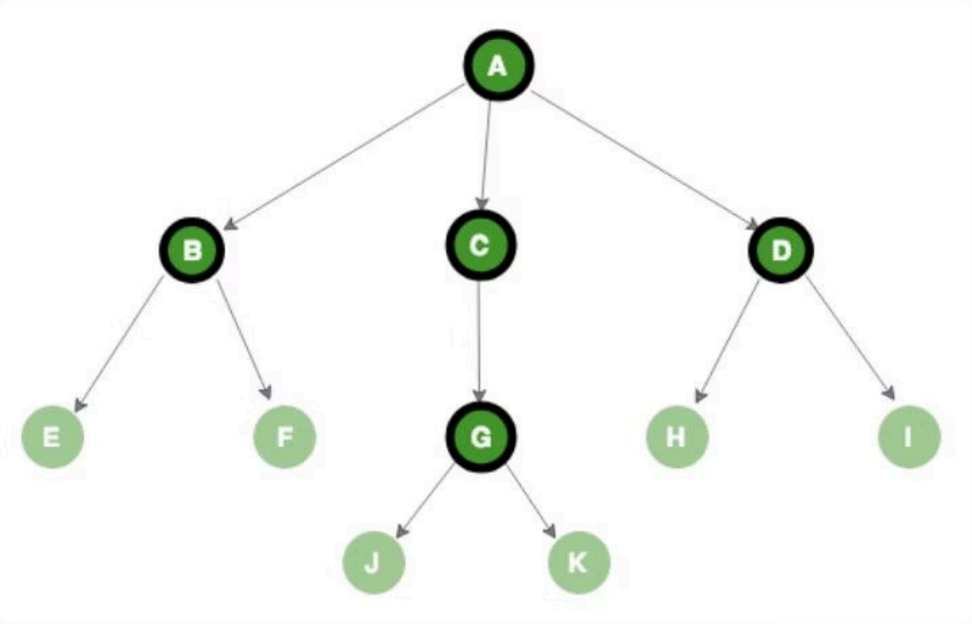
Düğüm Özellikleri

Dahili Düğüm

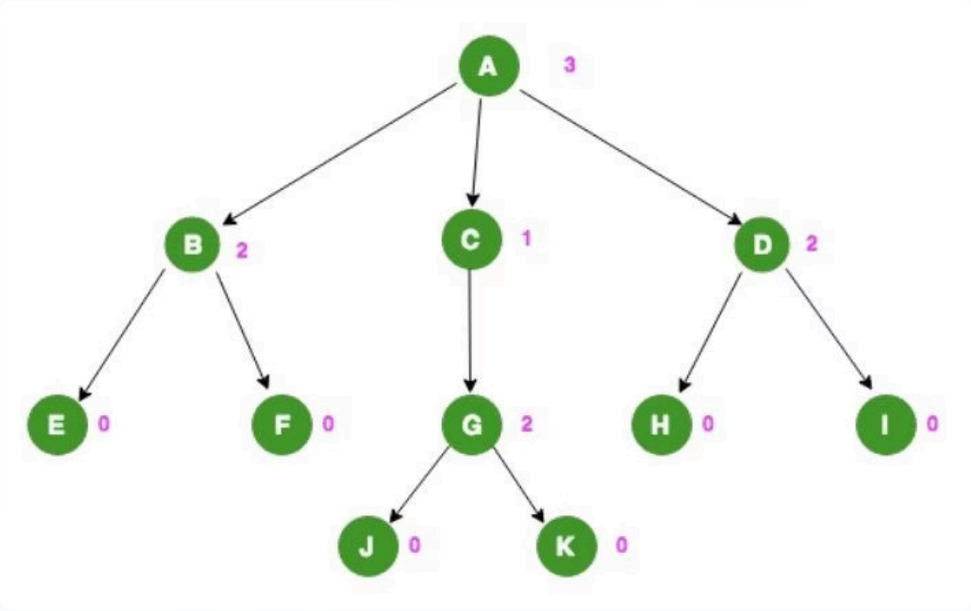
En az bir çocuğu olan düğüm. Yapraklar hariç tüm düğümler dahili düğümdür.

Degree (Derece)

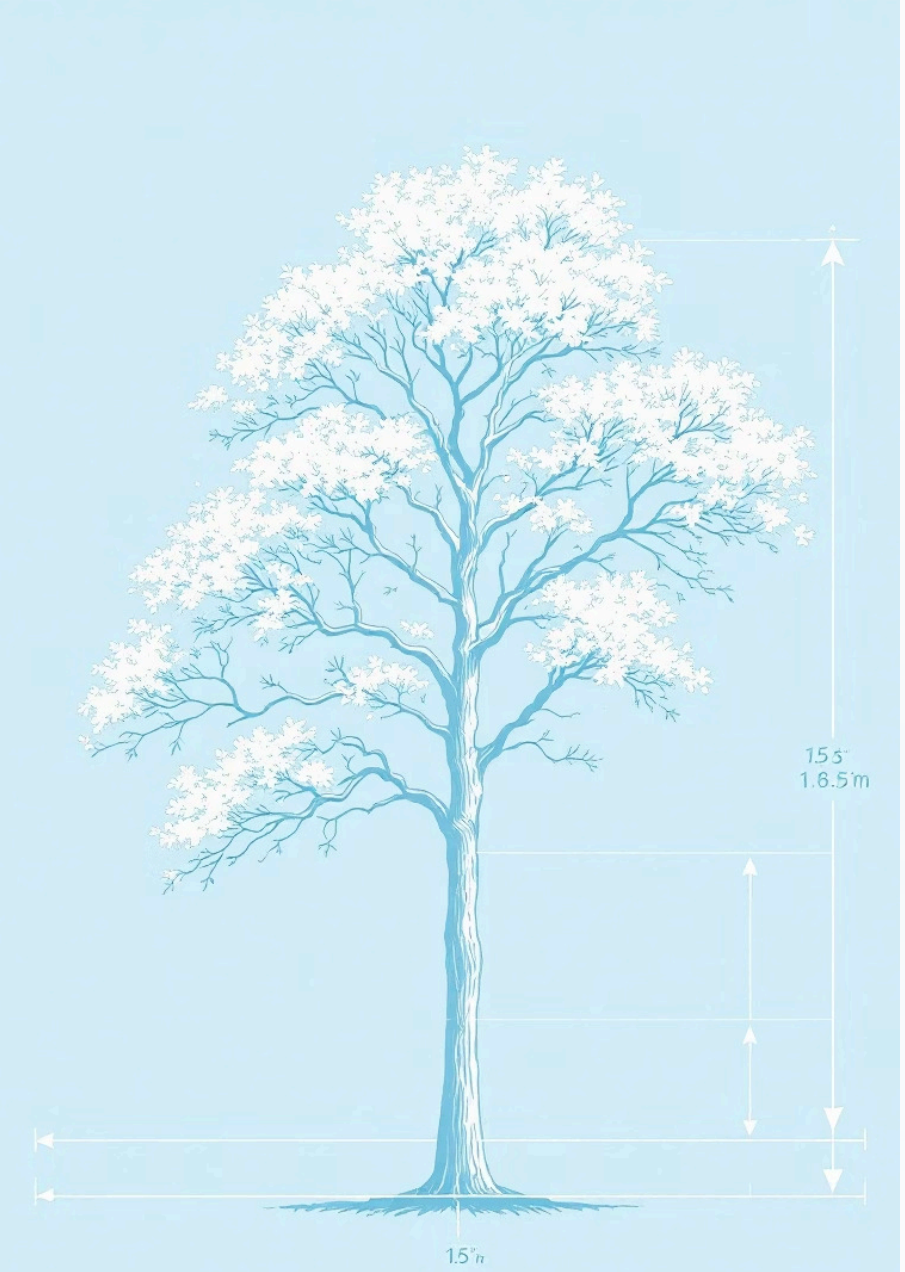
Bir düğümün alt hiyerarşiye yapılan bağlantı sayısı: çocuk veya alt ağaç sayısı. Yaprak düğümlerin derecesi: 0



Dahili Düğüm Örneği: Çocukları olan tüm düğümler



Derece Örneği: Tüm düğümlerin derecesi eşit ağaçlara homojen ağaçlar denir



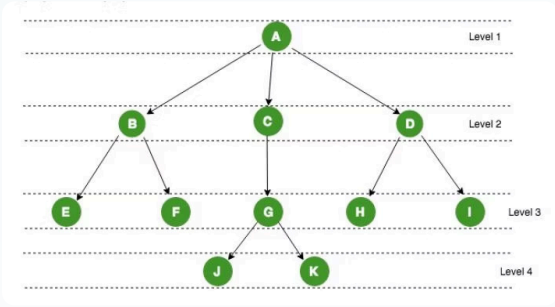
Seviye ve Yükseklik

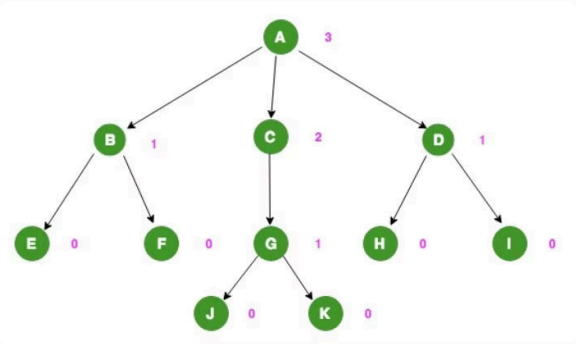
Level (Seviye)

Bir düğüm ile kök arasındaki edge sayısı + 1. Level 1'den başlar (kök). Children: Level 2

Height (Yükseklik)

Bir düğümden bir yaprağa olan en uzun yoldaki edge sayısı. Kök düğümün yüksekliği = Ağacın yüksekliği



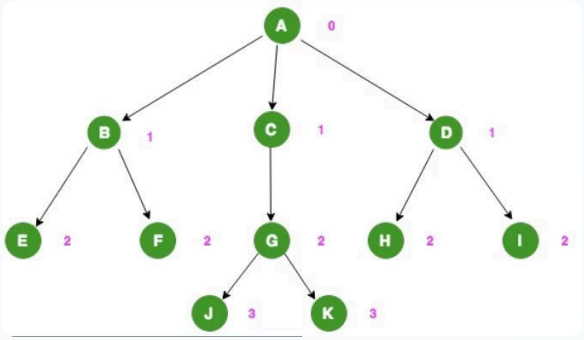


Derinlik ve Alt Ağaç

Depth (Derinlik)

Bir düğüm ile kök düğüm arasındaki edge (kenar) sayısı.

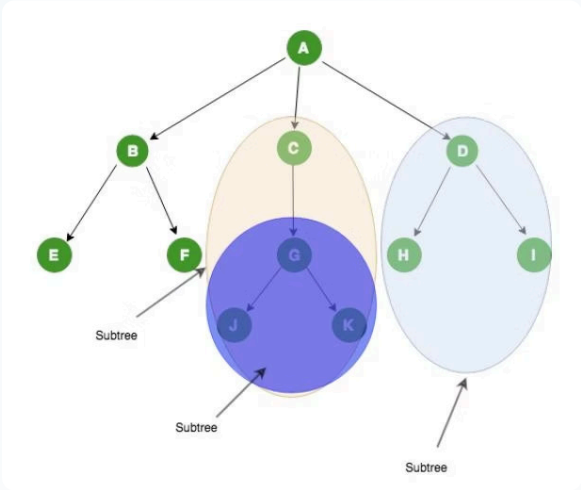
- Kök düğümün derinliği: 0
- Ağacın derinliği = En uzun yol sonundaki yaprak düğümün derinliği

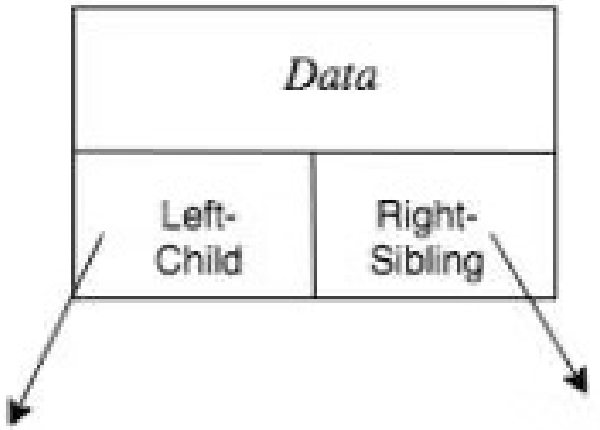


Subtree (Alt Ağaç)

Bir düğüm ile torunları tarafından oluşturulan ağaçlar.

Her düğüm kendi alt ağacının kökü olarak düşünülebilir.





Ağaçların Gösterimi

Ağaç veri yapıları bellekte farklı şekillerde temsil edilebilir. Gösterim yöntemi, ağacın tipine ve kullanım amacına göre değişir.

Bağlı Liste Gösterimi

Her düğüm, veri ve çocuklarına işaretçiler içerir. Esnek yapı, dinamik bellek kullanımı.

Dizi Gösterimi

Düğümler dizi elemanları olarak saklanır. Sabit bellek, hızlı erişim avantajı.

İşaretçi Tabanlı

Parent ve child işaretçileri kullanılır. Çift yönlü gezinme imkanı sağlar.